



Building a Material Science Data Intelligence Platform

ELT Pipeline · Relational Data Modeling · Open-Source EDA

May 2025



Context & Scope



Exercise Only

Decisions optimized for demonstrating reasoning over production use.



AI-Assisted

Domain research & implementation; insights are illustrative facts.



Public Data

Materials Project & Matminer (peer-reviewed) data sources used.

```
// DATA_SOURCES: MATERIALS_PROJECT, MATMINER  
// VERIFICATION_STATUS: ILLUSTRATIVE_ONLY
```



Deliverables: What Was Planned

- 01 Domain & data source research
- 02 Data acquisition pipeline (REST API + Python libs)
- 03 Data cleansing & normalisation
- 04 Relational star schema (formula ↔ material ↔ properties)
- 05 Chemical formula standardisation + FK creation
- 06 Reproducible ELT pipeline (Docker + justfile)
- 07 5 EDA dashboards · 20+ charts (Apache Superset)
- 08 Python code quality (Pydantic v2, mypy, Ruff)

```
// PIPELINE_STATUS: ALL_DELIVERABLES_COMPLETED  
// TIMEFRAME: MAY_2025_PROJECT_CYCLE
```



Research Methodology & Domain

"Research first. Code second."

Domain Research → Data Scoping → ELT Design → EDA Planning → Build

What was studied in material science:

Concept	Why it drove the design
Crystal systems	Geometry → properties; dashboard dimension
Energy Above Hull	0 = synthesisable; primary stability filter
Band Gap	Conductivity proxy: Metal vs Semiconductor
Formula ≠ Material	Carbon → diamond vs graphite; separate tables
DFT / PBE	Comp. method; validated by experiment
VRH averaging	Tensor to scalar; plottable poly_total

Key dataset decision:

Method	Data acquired
REST API	mp-api (Materials Project) 154k materials, core properties
Python lib	matminer datasets Dielectric, elastic, battery, steel

All raw data is **persisted to CSV before transformation**
— replay safety if the API changes.

⚠ AI was used as a domain research accelerator. Insights are illustrative — not validated scientific conclusions.

```
// DATA_SOURCES: MATERIALS_PROJECT, MATMINER
// STATUS: RESEARCH_PHASE_VALIDATED
```



Data Acquisition & ELT Architecture

"Simple ingestion now. Production-replaceable by design."

Exercise → Production mapping:

Exercise (Current)	Production Equivalent
REST API + Lib	Airflow / Kafka Connector
Local CSV	AWS S3
SQLite	AWS RDS / Redshift
Docker Compose	Kubernetes / Databricks
Manual trigger	Spark

ELT vs ETL Methodology:

ETL	ELT (Current Approach)
Transform before load	Load raw → transform at query
Data lost after transform	Raw always preserved
E: mp-api + Matminer → raw CSV L: raw CSV → SQLite T: Python & SQL Virtual Datasets (Superset)	

💡 "The pattern is designed to be replaced. The ingestion logic doesn't change — the infrastructure does."

```
// ARCHITECTURE: ELT_OVER_ETL  
// DESIGN_PATTERN: REPLACEABLE_INFRASTRUCTURE
```



Engineering Practices, DevOps & Challenges

"Good engineering decisions – even in an exercise."

Data Engineering

- **Pydantic v2:** Validates every API response at the boundary.
- **Domain-aware cleansing:** band_gap=0 kept as metal (47% of rows).
- **Star schema:** Formulas ↔ materials correctly models polymorphism.
- **Formula FK:** Normalization + memoized cache for JOIN keys.

Python & DevOps

```
just lint / format / typecheck # Ruff + mypy gates
```

- DB mounted in **Superset**; analytics cannot corrupt source.
- Named volumes ensure dashboards survive container restarts.
- Zero hardcoded credentials; all secrets managed via .env.

Top Challenges

Challenge	Resolution
New domain from scratch	AI-accelerated research + primary scientific sources.
e_total tensor collision	Found via EDA; switched to scalar poly_total.
Superset ECharts JSON	Complex charts finalized in UI. ECharts undocumented
Formula JOIN failure	Pymatgen normalization + memoized composition cache.

```
// REQ: QUALITY_DELIVERED  
// ENV: DOCKER_PROD_READY
```



EDA: Living Tool + Key Findings

"Not a report – a shared analytical workspace for three types of users."

User Personas

User	How they use it
Data Engineer	Validate pipeline output — distributions, FK joins.
Data Scientist	Feature exploration, scoring formula iteration.
Materials Expert	Scientific hypothesis testing, candidate screening.

5 Dashboards: Progressive Funnel

D1: Universe (154k) → "What do we have?"
D2: Battery Lens → "Candidates?"
D3: Specialty → "Optics & Ferroelectrics"
D4: Mechanical → "Strongest materials"
D5: AI Engineering → "Top 100 ranked"

EDA as Validation (Findings)

Findings that changed engineering

47% band_gap = 0: **scoring formula** v1 rejected due to ∞ scores for metals.

e_total tensor: switched to scalar poly_total after collision detected in EDA.

DFT Bias: Underestimates gaps; quantified bias before ranking candidates.

// TOOL: SUPERSET_BI
// INSIGHT: VALIDATION_DRIVEN



Live Demo: Apache Superset

"Let the data speak."

Dashboard 1 — Material Landscape

KPIs: 154k materials · 120k formulas · 7 systems

Distribution: Monoclinic dominates crystal landscape.

Stability: 22% sit at hull = 0 (synthesisable candidates).

Dashboard 2 — Battery Intelligence

Conductivity: Bottom-left is the high-performance target zone.

Band Gaps: DFT underestimates; error accounts for model bias.

Dashboard 5 — AI Workbench

Top 100 Candidates: Ranked via composite AI scoring.

Feature Engineering: Derived columns for model training.

"The dashboards are a living tool – real-time filtering, SQL sharing, and the base for the AI layer."

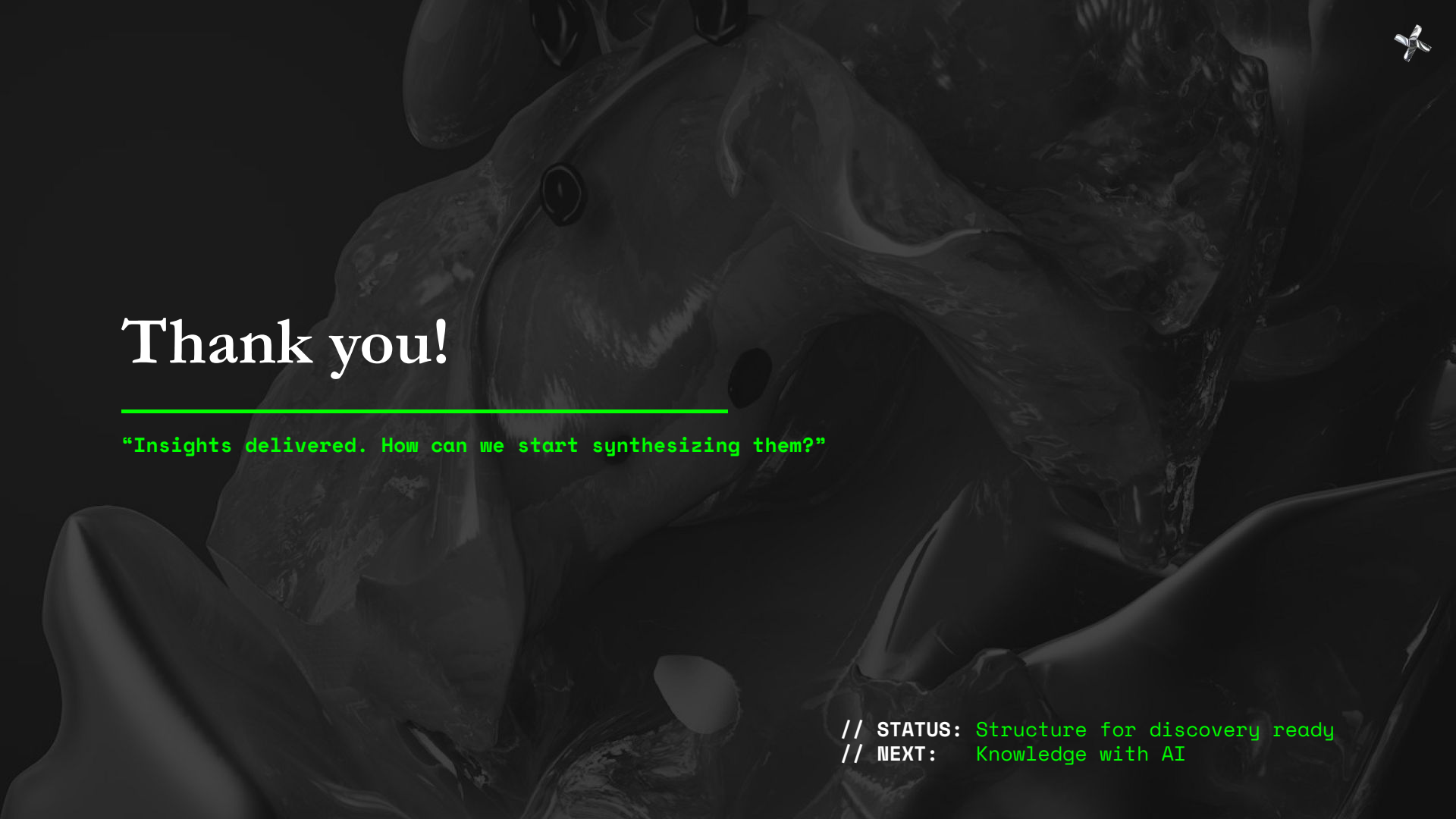
// URL: <http://localhost:8088>
// STATUS: DEMO_READY_LIVING_TOOL



Deliverables: What Was Delivered

"Good engineering decisions – even in an exercise."

#	Deliverable	Status
1	Domain & data source research	✓
2	Data acquisition pipeline (REST API + Python libs)	✓
3	Data cleansing & normalisation	✓
4	Relational star schema (formula ↔ material ↔ properties)	✓
5	Chemical formula standardisation + FK creation	✓
6	Reproducible ELT pipeline (Docker + justfile)	✓
7	5 EDA dashboards · 20+ charts (Apache Superset)	✓
8	Python code quality (Pydantic v2, mypy, Ruff)	✓



Thank you!

“Insights delivered. How can we start synthesizing them?”

// STATUS: Structure for discovery ready
// NEXT: Knowledge with AI



Appendix — Discussion Ready

"Supplementary materials for deep-dive sessions."



Appendix A — Research-First Methodology (detail)

"The project followed a strict phase-gated approach before any code was written."

PHASE 1

Domain Research

Understanding material science and computational informatics foundations. Defining key physical properties and scientific questions.

PHASE 2

Data Scoping

- Identifying reliable public datasets. Determining realistic exercise scope and explicit deferrals.

PHASE 3

ELT Design

- Structuring relational data models. Balancing raw loading vs. query-time transformation.

PHASE 4

EDA Planning

- Mapping scientific hypotheses to dashboard tests and appropriate chart visualizations.

PHASE 5

Implementation

- Pipeline development, data modeling, dashboard creation, and code quality enforcement.

Appendix B — Domain Research Detail

"Scientific foundations mapped to architectural requirements."

Concept	What was learned	Impact on design
Crystal systems	7 geometric families determine material behavior	Dimension in all dashboards
Energy Above Hull	0 = on the thermodynamic convex hull = synthesisable	Primary stability filter
Band Gap	0=metal, 1-3eV=semiconductor, >4eV=insulator	Drives candidate filter ('is_metal')
Formula ≠ Material	Same formula → multiple structures (polymorphism)	Forced 1-to-many DB model
DFT / PBE	Method that systematically underestimates band gap	Exp vs computed cross-validation
VRH averaging	Reduces a 3x3 dielectric tensor to one scalar	`poly_total` used for plotting

⚠ All material science domain knowledge was AI-accelerated. Insights are illustrative — not verified scientific conclusions.



Appendix C — Data Acquisition: Full Detail

"End-to-end provenance for materials informatics datasets."

REST API

mp-api (Materials Project)

- Authenticates via API key from .env
- Paginates automatically through 154k records
- Returns structured Python objects
→ validated by Pydantic before processing

PYTHON LIBRARY

matminer.datasets

- Downloads pre-curated, peer-reviewed ML-ready DataFrames
- No crawling or scraping required
- Data maintained by Lawrence Berkeley National Lab

PERSISTENCE

Raw Data Pattern

- All downloads saved as CSV files before any transformation
- Raw data preserved for replay if API or datasets update
- Eliminates redundant network calls and dependency on live endpoints



Appendix D — Data Ingestion: Pydantic Validation

"Strict schema enforcement at the system boundary."

PYDANTIC SCHEMA

```
class MaterialData(BaseModel):  
    material_id: str  
    density: float  
    band_gap: float  
    is_metal: bool  
    formation_energy: Optional[float] = None  
    energy_above_hull: Optional[float] = None  
    crystal_system: Optional[str] = None
```

VALIDATION LOGIC

- Type safety: Converts "N/A" strings to `ValidationError`s immediately.
- Null handling: Forces explicit `None` for missing optional fields.

INGESTION PRACTICES

- Security: API keys loaded via `.env` only.
- Portability: Runs inside Docker to eliminate environment drift.



Appendix E — Data Cleansing: Full Table

"Domain knowledge must drive cleansing decisions."

Issue	Scale	Action	Scientific reasoning
<code>`band_gap = 0`</code>	72,640 (47%)	✓ Kept	Zero gap = metal. Perfectly valid physically.
<code>hull = 0</code>	33,973 (22%)	✓ Kept	Hull = 0 = globally stable phase. Most valuable result.
<code>`e_total` tensor</code>	All dielectric	✓ Serialised	Tensor cannot be plotted — scalar VRH average used.
Formula notation	Cross-dataset	✓ Normalised	Enables JOIN across all 6 datasets via pymatgen.
Steel dataset ID	All rows	✓ Standalone	No MP material_id is available for alloy compositions.

****In a naive pipeline, ``band_gap = 0`` would be treated as missing data and removed —> silently deleting 47% of the dataset. **Domain knowledge** must drive cleansing decisions.****



Appendix F — Data Architecture & Modelling: Full Star Schema

"The polymorphism problem: Chemical identity ≠ Physical structure."

THE POLYMORPHISM PROBLEM

Formula maps to multiple `material_id` entries:

- **mp-66 (Diamond):** Hardest natural material, insulator
- **mp-48 (Graphite):** Conductor, lubricant

KEY ARCHITECTURAL DECISIONS

- **Entity Separation:** `formulas` table defines chemical identity; `materials` defines physical structure.
- **Stable Joins:** All property tables join via `material_id` - a single immutable FK throughout the schema.
- **Steel Isolation:** `experiments_steel` remains standalone; forcing a join would introduce false data artifacts.

1-TO-MANY STAR SCHEMA

FORMULAS

(1 row per unique canonical formula)

MATERIALS

(N rows per formula, polymorphic)

properties_core

Density, Gap, Stability

prop_dielectric

ϵ_{total} tensor, n

properties_elastic

K_VRH, Poisson, Anis.

exp_battery

Exp. gap measurements

structures_mp_gap

DFT-computed validation

experiments_steel
Standalone (No FK)



Appendix G — Formula Standardisation: Full Implementation

"Ensuring data joinability through chemical normalization and memoization."

THE CROSS-DATASET PROBLEM

A JOIN on raw strings returns **0 matches** due to notation variance:

"Fe203" ← Matminer notation
"Fe01.5" ← Alternative ratio
"Fe₀," ← Unicode subscript

WHY MEMOIZE?

- **Parsing Overhead:** `Composition()` is an expensive call involving complex chemical parsing logic.
- **Data Density:** With 154k rows and high repetition, O(1) cache hits eliminate redundant computation.
- **ETL Pattern:** Implements a practical performance pattern essential for bulk data pipelines.

FULL IMPLEMENTATION

```
class DataTransformer:

    def __init__(self):
        self._cache: Dict[str, str] = {}

    def standardize(self, raw: str):
        if raw in self._cache:
            return self._cache[raw] # O(1) hit
        try:
            res = Composition(raw).reduced_formula
            self._cache[raw] = res
            return res
        except Exception:
            return raw # Fallback
```



Appendix H — EDA Planning: Objectives-First Design

"Each dashboard was specified in 'PLAN_data_exploratory_dashboard.md' before Superset was opened:"

THE PLANNING HIERARCHY

1. **Scientific/business question:** Define core hypothesis first.
2. **Data Logic:** Primary tables and specific JOIN requirements.
3. **Visual fit:** Chart type selection based on the data relationship.
4. **Detailed Mapping:** Exact axes, dimensions, and metrics.

INTERPRETATION GUIDE

- **Diagonal Check:** Points on $y=x$ indicate an accurate predictive model.
- **Systematic Bias:** Upward offset implies DFT underestimates (expected PBE functional behavior).
- **Structural Failures:** Outliers far off diagonal highlight crystal types where the model breaks down.

SPECIFICATION: DASHBOARD 2

Question: Is our DFT model trustworthy?
Chart: Scatter Plot (Continuous vs Continuous)

Dataset: Virtual JOIN
experiments_battery + structures_mp_gap
ON formula

X_axis: gap_pbe (DFT-calculated)
Y_axis: gap_expt (Measured)

Analysis Criteria:
→ $y=x$ diagonal == High Fidelity
→ Systemic Offset == PBE Underestimation
→ Distance Off-Diagonal == Model Failure



Appendix I — EDA: Multi-Team Usage Detail

"Superset acts as the central interface for cross-functional scientific validation."

USER TYPE	ACTION	SPECIFIC EXAMPLE
Data Engineer	Validate pipeline — check distributions, confirm FK joins, verify null handling	Stability Landscape: confirms hull values are correctly distributed; no anomalous negatives
Data Scientist	Explore features, iterate on scoring formulas, test distributions	Band Gap histogram: revealed 47% zeros — changed scoring formula entirely
Materials Expert	Test scientific hypotheses, screen candidates, compare with materials	Battery dashboard: filter crystal system, compare experimental vs. DFT band gaps

WHY APACHE SUPERSET?

SQL-native: Any team member with SQL knowledge can create their own chart without UI friction.

Virtual Datasets: Complex JOINS are encapsulated and reusable across teams without redundancy.

Open-source: No vendor dependency, no licensing cost, full auditability of the data tool itself.



Appendix J — Python & DevOps: Full Detail

"Three automated quality gates and strict containerization practices."

QUALITY GATES & TYPING

```
# Automated Quality Gates
just lint # ruff check src --fix
just format # ruff format src
just typecheck # mypy src

# Type Annotations
def standardize_formula(self, raw: str) -> str:
    self._cache: Dict[str, str] = {}
```

DEV WORKFLOW (justfile)

```
download # Fetch raw datasets
import # Run ETL pipeline
eda-up # Start Superset stack
eda-down # Stop the stack
lint # Code quality check
format # Auto-format source
```

PATTERN

IMPLEMENTATION

WHY

Read-only mount

`materials.db :ro`

Analytics layer cannot corrupt source of truth.

Named volume

`superset_home`

Dashboards persist across container restarts.

Health-check

`curl //health init`

Prevents bootstrap race conditions on startup.

Single compose

`docker-compose.yml`

One command to reproduce the entire stack.

Env-var secrets

`.env injection`

No hardcoded secrets in the repository.



Appendix K — Challenges: Full Detail

"The most interesting challenges were at domain boundaries — where material science meets data engineering."

CHALLENGE	EFFORT	RESOLUTION
Learning material science from scratch	High	AI-accelerated domain research + Materials Project documentation + Matminer papers
e_total is a 3×3 tensor, not a float	Medium	Discovered via EDA when Superset returned 0 for all values; switched to scalar poly_total
Superset ECharts JSON is undocumented	High	ORM bootstrap scripts generate the boilerplate; complex charts finalized once in UI
Formula JOIN fails silently across datasets	Medium	All 4 variants resolved via pymatgen.Composition.reduced_formula + memoized cache
SQLite metadata conflict	Medium	Separated via Docker named volume for Superset; materials.db mounted :ro
Scoring formula broke on 47% zero band gaps	Low	Caught immediately via EDA; replaced 1/band_gap with min-max normalised formula



Appendix L — Scoring Formula: Two Iterations

"From raw reciprocal formulas to min-max normalized semantic scoring peaks."

VERSION 1 — REJECTED

UNSTABLE

```
-- Reciprocal distance formula
material_score =
(1 / NULLIF(density, 0)) * 0.3 +
(1 / NULLIF(band_gap, 0)) * 0.3 +
(1 / NULLIF(energy_above_hull, 0)) * 0.4
```

Why rejected:

47% of **band_gap** values are exactly 0 (metals).

The **1/0** logic leads to infinite scores for all metallic structures.

Resulted in metals being ranked #1, which is completely wrong for battery cathode selection.

VERSION 2 — ADOPTED

IMPLEMENTED

```
-- Min-max normalized approach
(1.0 - (density / 26.58)) * 0.3 -- lightweight
+ (1.0 - ABS(band_gap - 1.5) / 17.89) * 0.3 -- sweet spot
+ (1.0 - (energy_above_hull / 9.71)) * 0.4 -- stable
```

Technical Improvement:

- Normalized using known dataset ranges → scores [0, 1].
- **band_gap = 1.5 eV** is the peak (optimal semiconductor).
- Metals score 0.79 on conductivity term but excluded via filter.



Appendix M — Superset Automation Limitation

"Architectural constraint where programmatic ORM sync meets React-based visual state."

PROGRAMMATIC SYNC (ORM)

```
# What works via Superset ORM
table = SqlaTable(table_name="Virtual:...", database=db)
db.session.add(table)
table.fetch_metadata() # syncs columns
db.session.commit()
```

RESOLUTION WORKFLOW

- **ORM scripts** create chart shell and register dataset.
- Manual **"Edit Chart" → "Update Chart"** finalizes JSON.
- Resulting chart becomes persistent/exportable.

UI FINALIZATION REQUIREMENT

The "Opaque JSON" Problem:

Modern ECharts-based visualizations (scatter, histogram, bubble) store config as a React-generated payload that cannot be reliably constructed programmatically.

Limitation Drivers:

- Schema is not publicly documented.
- Contains version-specific React component state.
- Minor JSON discrepancies cause silent chart failure.

This is an architectural limitation of Superset — not an implementation gap.



Appendix N — Scientific Glossary

"Standardizing domain terminology across materials science datasets and visualization layers."

TERM	PLAIN ENGLISH	USED IN
Energy Above Hull (eV/atom)	"How far from the most stable known structure — 0 = real-world material"	Dashboards 1, 2, 3, 5
Formation Energy (eV/atom)	"How strongly the material wants to hold together"	Dashboard 1
Band Gap (eV)	"How conductive the material is"	All dashboards
poly_total	"Scalar dielectric constant — ability to store charge"	Dashboard 3
K_VRH / G_VRH (GPa)	"Resistances to compression / shearing"	Dashboard 4
Elastic Anisotropy	"Direction-dependent mechanical response — 0 = uniform"	Dashboard 4
Poisson Ratio	"Narrows when stretched — 0.3 = standard metals"	Dashboard 4
Ferroelectric	"Can flip electrical state — FeRAM memory and sensors"	Dashboard 3
Polymorphism	"Same formula, different crystal structure/properties"	Data model
DFT / PBE	"Physics simulation — fast but underestimates band gap"	Dashboard 2
VRH averaging	"Voigt-Reuss-Hill — reduces 3D tensor to scalar"	Dashboard 3